Sistemas Operativos

Práctica 3: Scheduler. Guía de estudio teórica: planificación de CPU, expropiación, Round Robin y Timer

Esta guía junta toda la teoría que necesitás para el TP 3. Se apoya en lo que ya vimos de la práctica de multiprogramación (Dispatcher, PCB, handlers, interrupciones) y agrega lo nuevo: el Scheduler, los algoritmos, la expropiación y el Timer. No incluye código; el objetivo es que entiendas qué hay que construir y por qué.

Contenido

- **Temas obligatorios del TP**, de un vistazo
- **Lo que necesitás de antes**: repaso mínimo
- **Bloque A.** El Scheduler
- **Bloque B.** Los algoritmos: FCFS, Priority (no expropiativo y expropiativo), Round Robin y Aging
- **Bloque C.** Expropiación
- **Bloque D.** Timer, quantum y la interrupción #TIMEOUT
- **Bloque E.** Prioridad en *run*, cambio de scheduler en frío y estadísticas (Gantt)
- **Qué hay que implementar** (checklist), autoevaluación y glosario

Temas obligatorios del TP

Esto es lo que pide la consigna, y a qué bloque de esta guía corresponde cada cosa:

#	Tema	Por qué es obligatorio	Bloque
1	Scheduler como componente	Es el único que administra la Ready Queue	A
2	FCFS	Algoritmo pedido	B
3	Priority no expropiativo	Algoritmo pedido	B
4	Priority expropiativo	Algoritmo pedido	B y C
5	Round Robin	Algoritmo pedido	B y D
6	Aging	Pedido en los schedulers por prioridad	B
7	Expropiación (*mustExpropiate*)	Necesaria para Priority expropiativo	C
8	Timer, quantum y #TIMEOUT	Necesarios para Round Robin	D
9	Reset del Timer en el context switch	Para que cada proceso arranque con la cuenta en 0	D
10	Prioridad en *run* y en #NEW	Priority necesita saber la prioridad de cada proceso	E
11	Cambiar de scheduler "en frío"	Pedido explícitamente	E
12	Estadísticas (#STAT) y Diagrama de Gantt	Deseable, suma puntos	E

Lo que necesitás de antes (repaso mínimo)

- **Dispatcher**: *load(pcb)* pone un proceso en la CPU (PC del PCB a la CPU, baseDir a la MMU) y *save(pcb)* lo saca (PC de la CPU al PCB, CPU idle con PC = -1). Es el que hace el context switch.
- **PCB y runningPCB**: el PCB es la ficha del proceso (pid, baseDir, pc, estado, nombre). El kernel guarda cuál es el que está en la CPU. Si *runningPCB* es vacío, la CPU está libre.
- **Estados**: new, ready, running, waiting, terminated.
- **Los 4 handlers**: #NEW y #IO_OUT hacen entrar a un proceso (a la CPU si está libre, a ready si no). #IO_IN y #KILL liberan la CPU y buscan reemplazo.
- **IRQ**: tiene un *type* (qué pasó, lo usa el Interrupt Vector para elegir el handler) y *parameters* (los datos, los usa el handler).
- **Tick y Clock**: el Clock avisa en cada tick a los componentes; todo lo que pasa en el emulador pasa dentro de un tick.

Bloque A · El Scheduler

A.1 El problema

Con multiprogramación puede haber varios procesos en estado **ready** esperando la CPU, pero la CPU ejecuta uno solo. Cada vez que se libera (un proceso termina, se va a I/O o lo desalojan), alguien tiene que decidir **cuál de los que esperan entra ahora**. Eso es la **planificación (scheduling)**, y la regla con la que se decide es el **algoritmo de planificación**. En la práctica anterior la regla era fija: el primero que llegó. Ahora la vamos a poder elegir.

A.2 Qué es el Scheduler

Es el componente del S.O. que **administra la Ready Queue**: guarda los procesos listos y decide cuál sale primero.

> **Idea clave**
> **El Scheduler es el único componente que accede y modifica la Ready Queue.** Los handlers, el Kernel y el Dispatcher no la tocan: le piden cosas al scheduler.

A.3 El protocolo común (la interfaz)

Todos los schedulers tienen que ofrecer las mismas operaciones, con el mismo nombre. Los nombres exactos los define tu equipo; estos son los que usa la slide:

Operación	Qué hace	Quién la usa
add(pcb)	Mete un proceso que acaba de pasar a ready	#NEW, #IO_OUT, #TIMEOUT
getNext()	Saca y devuelve el proceso que debe correr ahora	#IO_IN, #KILL, #TIMEOUT
isEmpty()	Dice si no hay nadie esperando. Hay que chequearlo antes de pedir el próximo	#IO_IN, #KILL, #TIMEOUT
mustExpropiate(pcbInCPU, pcbToAdd)	Dice si el que llega debe sacarle la CPU al que corre (Bloque C)	#NEW, #IO_OUT

A.4 Por qué se esconde la Ready Queue

Si un handler hiciera "agregar a la cola" directamente, quedaría atado a una cola FIFO. En cambio, si le pide al scheduler que agregue, **no sabe cómo se ordena adentro**:

- En **FCFS**, agregar es poner al final.
- En **Priority**, agregar es ubicarlo según su prioridad.
- En **Round Robin**, agregar es poner al final y el quantum se maneja aparte.

Así, **cambiar de algoritmo es cambiar la clase del scheduler** y los handlers no se tocan. Eso es polimorfismo: varias clases con los mismos métodos, intercambiables. En la práctica se arma una clase base con la interfaz y una subclase por algoritmo.

A.5 Qué cambia en los handlers

La lógica de la práctica anterior se mantiene. Cambia con quién hablan:

Handler	Antes	Ahora
#NEW	CPU libre: cargar. Ocupada: encolar en la Ready Queue	CPU libre: cargar. Ocupada: preguntar al scheduler si expropia; si no, *add*

Handler	Antes	Ahora
#IO_OUT	Igual que #NEW	Igual que #NEW (consulta *mustExpropiate* o *add*)
#IO_IN	Guardar el proceso, mandarlo a I/O, sacar el próximo de la cola	Igual, pero el próximo sale de *getNext()* (si *isEmpty()* es falso)
#KILL	Terminar el proceso y sacar el próximo de la cola	Igual, con *getNext()*
#TIMEOUT (nuevo)	No existía	Solo para Round Robin (Bloque D)

Ojo
getNext() sobre una cola vacía da error. Siempre se pregunta *isEmpty()* primero.

Bloque B · Los algoritmos de planificación

B.1 Dos conceptos antes de empezar

- **Expropiativo (preemptive):** el S.O. **puede sacarle la CPU** a un proceso que está corriendo, aunque no haya terminado ni pedido I/O.
- **No expropiativo:** el proceso **se queda con la CPU** hasta que termina o se va a I/O.

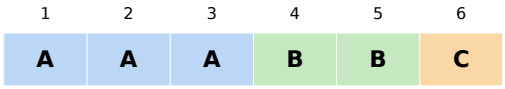
B.2 Ejemplo base

Para ver las diferencias uso tres procesos que solo usan CPU (sin contar el EXIT): **A** necesita 3 ticks, **B** necesita 2 y **C** necesita 1. Llegan en el orden A, B, C.

B.3 FCFS (First Come, First Served)

Se atiende en **orden de llegada**, sin expropiar. La Ready Queue es una cola FIFO simple.

FCFS



- **Ventaja:** simple y justo en el orden.
- **Problema:** si el primero es largo, los cortos esperan mucho (**efecto convoy**). C necesita 1 tick y espera 5.

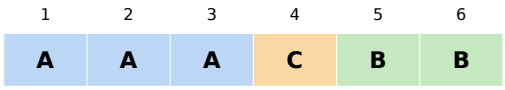
B.4 Priority no expropiativo

Cada proceso tiene una **prioridad**. Cuando la CPU se libera, sale de la cola **el de mayor prioridad**. Pero al que ya está corriendo nadie lo desaloja.

> **Convención de prioridades**
> Hay que fijar una convención: lo más común es que **un número menor sea prioridad mayor** (1 es la más alta). Verificá con tu cátedra cuál usan, porque algunos usan lo inverso, y mantenela igual en todo el TP.

Ejemplo: prioridades A = 2, B = 3, C = 1. C llega cuando A ya corrió 2 ticks.

Priority no expropiativo

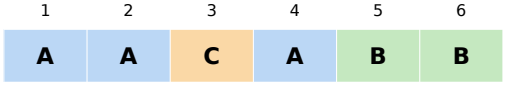


C tiene mejor prioridad que A, pero **espera a que A termine**. Recién ahí compite con B y gana.

B.5 Priority expropiativo

Igual, pero si llega un proceso **de mayor prioridad que el que está corriendo**, se la saca en el momento.

Priority expropiativo



C llega después del tick 2 con mejor prioridad que A, así que lo expropia. A vuelve a ready con lo que le queda (1 tick) y C corre. Cuando C termina, A sigue donde se había quedado.

- **Ventaja:** el proceso importante no espera.
- **Problema:** la **inanición (starvation)**. Si siguen llegando procesos de mayor prioridad, los de baja prioridad nunca corren. De eso se encarga el aging.

B.6 Aging (envejecimiento)

Es la solución a la inanición: **cuanto más espera un proceso en la Ready Queue, más sube su prioridad**. Hasta el de menor prioridad termina llegando a correr. Se pide en los dos schedulers por prioridad.

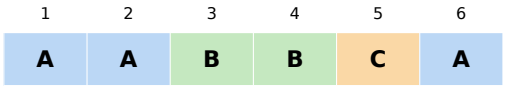
- Una forma simple: cada vez que el scheduler elige un proceso (*getNext*), a los que quedan esperando se les mejora la prioridad un nivel (o cada N ticks de espera). El criterio exacto es una decisión de diseño de tu equipo; conviene dejarlo como un parámetro.
- Lo implementa el **scheduler**, porque es el único que ve la Ready Queue.
- Cuidado con el límite: la prioridad no puede pasar de la más alta.

B.7 Round Robin

No usa prioridades. Cada proceso corre **como máximo un quantum** (un número fijo de ticks). Si no terminó, se lo **expropia** y va al final de la cola.

Ejemplo con quantum = 2:

Round Robin (q = 2)



A corre 2 ticks, se le acaba el quantum y va al final. B corre sus 2 ticks y termina. C corre 1 y termina. A vuelve y completa el tick que le faltaba.

- **Ventaja:** nadie espera demasiado; es el más justo.
- **Problema:** con quantum muy chico se pierde mucho en cambios de contexto; con quantum muy grande se parece a FCFS.

B.8 Comparación

Algoritmo	¿Expropia?	Elige por...	Riesgo
FCFS	No	Orden de llegada	Efecto convoy
Priority no expropiativo	No	Prioridad	Inanición (se resuelve con aging)
Priority expropiativo	Sí, al llegar uno mejor	Prioridad	Inanición (se resuelve con aging)
Round Robin	Sí, por quantum	Turnos	Quantum mal elegido

Bloque C · Expropiación

La slide plantea: *¿cómo podemos manejar la expropiación?* La respuesta es una regla: **cada vez que un PCB pasa a ready y la CPU está ocupada, hay que preguntarle al scheduler si debe expropiar**.

C.1 El flujo

Llamemos *pcbToAdd* al PCB que acaba de pasar a ready y *pcbInCPU* al que está corriendo:

- Se pregunta *scheduler.mustExpropiate(pcbInCPU, pcbToAdd)*.
- **Si es verdadero:** se saca *pcbInCPU* de la CPU (Dispatcher *save*), pasa a ready y se agrega al scheduler; luego se carga *pcbToAdd* en la CPU y pasa a running.
- **Si es falso:** *pcbToAdd* se agrega a la cola de ready del scheduler y el que corre sigue corriendo.

C.2 Quién responde qué

Scheduler	***mustExpropiate* devuelve...**
FCFS	Siempre falso
Priority no expropiativo	Siempre falso
Priority expropiativo	Verdadero si *pcbToAdd* tiene mejor prioridad que *pcbInCPU*
Round Robin	Siempre falso (expropia el Timer, no el que llega)

La ventaja de este diseño es que **los handlers hacen siempre lo mismo** (preguntar y actuar) y cada scheduler decide con su propia regla. Se usa en #NEW y en #IO_OUT, que son los dos momentos en que aparece un proceso nuevo en ready.

Ojo
Si la CPU está libre no hay nada que expropiar: se carga el proceso directamente, sin preguntar. Y al expropiar hay que actualizar el estado de los dos procesos (el desalojado pasa a ready, el nuevo a running) y el *runningPCB* del kernel.

Bloque D · Timer, quantum y Round Robin

D.1 El problema

Round Robin necesita cortar a un proceso cuando cumple su quantum. La slide pregunta: *¿tenemos control de cuántos ticks puede correr la CPU?* La CPU sola no lo tiene. Hace falta un componente de **hardware** nuevo.

D.2 El Timer

El Timer se pone **entre el Clock y la CPU**: el Clock le da el tick al Timer y el Timer se lo pasa a la CPU. Mientras tanto, cuenta cuántos ticks lleva corriendo el proceso actual.

- Si el Timer está **activo**, la cuenta llegó al quantum y la CPU está ocupada, lanza una interrupción **#TIMEOUT** al Interrupt Vector.
- Después suma un tick a la cuenta y le pasa el tick a la CPU.
- **Por defecto está desactivado.** Se activa al configurarle un quantum.
- Por eso, con FCFS o Priority simplemente no se configura el quantum y la CPU ejecuta todos los ticks sin cortes.

D.3 Context switch: el reset del Timer

La slide pregunta: *¿está completo? ¿qué pasa si el proceso de CPU se va a I/O antes de que se cumpla el quantum?*

Si el proceso se va a I/O a mitad de quantum, la cuenta del Timer queda a medias. El proceso siguiente heredaría esa cuenta y lo cortarían antes de tiempo. Por eso **en cada context switch hay que reiniciar la cuenta del Timer a 0**. Se hace en el Dispatcher, en *load*: cada vez que se carga un proceso en la CPU, se llama a *reset()* del Timer. Así todos los procesos arrancan con su quantum completo, venga como venga el cambio (por #IO_IN, #KILL, #TIMEOUT o expropiación).

> **Idea clave**
> **Dispatcher.load** = poner el PC en la CPU + el baseDir en la MMU + reiniciar la cuenta del Timer.

D.4 El handler de #TIMEOUT

Es una interrupción nueva, con su handler registrado en el Interrupt Vector (como las demás). Cuando se cumple el quantum:

- Se saca al proceso de la CPU (Dispatcher *save*), pasa a **ready** y se agrega al scheduler (queda al final de la cola).
- El kernel anota que la CPU quedó libre y se busca el próximo con *getNext()*; se carga y pasa a **running**.
- Caso borde (decisión de diseño): si nadie más espera, lo razonable es que el mismo proceso siga corriendo, porque no tiene sentido desalojarlo para volver a cargarlo.

D.5 Configuración de Round Robin

- Al armar el Kernel se registra el handler de #TIMEOUT y se crea el scheduler de Round Robin.
- Se configura el quantum del Timer (la slide usa 4). Se puede hacer desde el Kernel o, mejor, que lo configure el propio scheduler de Round Robin.
- Los demás schedulers no tocan el quantum: el Timer queda desactivado.

Bloque E · Prioridad en run, cambio en frío y estadísticas

E.1 La prioridad entra por run

Para Priority el sistema necesita conocer la prioridad de cada proceso. Cambia la llamada de ejecución: **run(programa, prioridad)**. El Kernel arma la IRQ #NEW con un **diccionario** como parámetro (*program* y *priority*) y el handler de #NEW lo desarma. El PCB tiene que tener un campo de prioridad, y es el que usa el scheduler (y el aging va modificando).

Prioridades de la consigna: prg1 = 1, prg2 = 2, prg3 = 3.

E.2 Cambiar de scheduler "en frío"

La consigna exige poder cambiar el algoritmo, pero solo **bajando el sistema y volviéndolo a levantar**, nunca en plena ejecución. Con la arquitectura del Bloque A es sencillo: al crear el Kernel se elige qué scheduler instanciar, y para probar otro se cambia esa elección y se ejecuta de nuevo desde cero.

E.3 Estadísticas y Diagrama de Gantt (deseable)

Suma puntos en la nota, pero no es obligatorio. La consigna da estas pistas:

- Se habilitan las estadísticas poniendo en verdadero el flag **enable_stats** de la CPU.
- Se puede agregar un handler para la interrupción **#STAT**.
- Para saber en qué tick estás, se le pregunta al Clock del hardware (*currentTick*).

La idea del **Diagrama de Gantt** es registrar, tick a tick, qué proceso estuvo en la CPU, y al final imprimirlo como una tabla con una fila por proceso y una columna por tick. Es lo mismo que los diagramas de tiempos de esta guía. No sé con certeza qué datos manda la CPU en cada IRQ #STAT; eso se ve en el *hardware.py* nuevo que acompaña la práctica.

Qué hay que implementar (checklist)

Componente	Qué hay que hacer
Scheduler (clase base)	Definir la interfaz: *add*, *getNext*, *isEmpty*, *mustExpropiate*. La Ready Queue vive adentro
FCFS	Cola FIFO; *mustExpropiate* siempre falso
Priority no expropiativo	Cola ordenada por prioridad; siempre falso; aging
Priority expropiativo	Igual, pero *mustExpropiate* compara prioridades; aging
Round Robin	Cola FIFO; siempre falso; configura el quantum del Timer
PCB	Agregar el campo de prioridad
Kernel	*run(program, priority)* con diccionario en la IRQ; guarda el scheduler elegido; registra el handler de #TIMEOUT; configura el quantum (solo RR)
#NEW	Leer el diccionario; crear el PCB con prioridad; CPU libre: cargar; si no, *mustExpropiate* o *add*
#IO_OUT	CPU libre: cargar; si no, *mustExpropiate* o *add*
#IO_IN y #KILL	Liberar la CPU y buscar el próximo con *getNext()* (chequeando *isEmpty()*)
#TIMEOUT	Desalojar, agregar al scheduler y cargar el próximo

Componente	Qué hay que hacer
Dispatcher.load	Además de PC y baseDir, reiniciar el Timer
main	Elegir el scheduler, pasar la prioridad a cada *run*; opcional: *enable_stats*

Para probar: ejecutá los tres programas de la consigna con cada scheduler y compará los resultados contra tu propio diagrama de tiempos hecho a mano. Si coinciden, la lógica está bien.

Autoevaluación y glosario

Pregunta	Respuesta
¿Qué hace el Scheduler?	Administra la Ready Queue y decide qué proceso de ready corre cuando la CPU se libera.
¿Por qué los handlers no usan la Ready Queue directamente?	Para poder cambiar de algoritmo sin tocarlos: solo conocen los métodos del scheduler.
¿Qué hay que chequear antes de *getNext()*?	Que el scheduler no esté vacío (*isEmpty()*).
¿Qué diferencia hay entre Priority expropiativo y no expropiativo?	En el expropiativo, si llega un proceso de mejor prioridad le saca la CPU al que corre; en el otro espera a que el actual termine o se vaya a I/O.
¿Qué es la inanición y cómo se evita?	Que un proceso de baja prioridad nunca corra. Se evita con aging: su prioridad mejora a medida que espera.
¿Quién decide si hay que expropiar?	El scheduler, con *mustExpropiate*. Los handlers solo preguntan y actúan.
¿En qué handlers se consulta la expropiación?	En #NEW y en #IO_OUT, cuando la CPU está ocupada.
¿Qué hace el Timer?	Cuenta los ticks del proceso actual y, al llegar al quantum, lanza #TIMEOUT.
¿Cuándo se desactiva el Timer?	Cuando no se le configura quantum (es lo que pasa con FCFS y Priority).
¿Por qué hay que reiniciar el Timer en cada context switch?	Para que cada proceso arranque con su quantum completo y no herede la cuenta del anterior.
¿Qué hace el handler de #TIMEOUT?	Desaloja al proceso, lo manda al final de ready y carga el próximo.
¿Cómo se cambia de scheduler?	En frío: se elige al crear el Kernel y se vuelve a ejecutar todo desde cero.

Glosario

Término	Definición
Scheduler	Componente que administra la Ready Queue y elige qué proceso corre.
Planificación	Decidir qué proceso de ready ocupa la CPU.
FCFS	First Come, First Served: se atiende por orden de llegada.
Prioridad	Número que indica qué tan importante es un proceso para el scheduler.
Expropiación	Sacarle la CPU a un proceso que todavía no terminó.
Inanición	Un proceso que nunca llega a correr por culpa de otros de mayor prioridad.
Aging	Mejorar la prioridad de los procesos que esperan, para evitar la inanición.

Término	Definición
Quantum	Máximo de ticks seguidos que un proceso puede usar la CPU en Round Robin.
Timer	Hardware que cuenta los ticks del proceso en CPU y lanza #TIMEOUT al cumplir el quantum.
#TIMEOUT	Interrupción que avisa que se cumplió el quantum.
Context switch	Cambio del proceso que ocupa la CPU por otro.
Diagrama de Gantt	Tabla que muestra qué proceso ocupó la CPU en cada tick.